



GUÍA DE ESTUDIO

BIBLIOTECA DE CÓDIGO

CHULETA

Kotlin

La guía definitiva para aprobar el examen

Teoría explicada paso a paso, código listo para copiar y adaptar, métodos para corregir y ampliar programas, errores típicos y una chuleta final para repasar en cinco minutos.

```
Examen.kt

fun main() {
    val temas = listOf("listas", "lambdas",
        "null safety", "clases", "Compose")
    var nota = 0.0
    for (tema in temas) {
        nota += 2.0 // tema dominado = +2
    }
    println("¡Aprobado con un $nota!")
}

> ¡Aprobado con un 10.0!
```

01

Entender

Cada concepto explicado: qué hace, por qué y qué falla.

02

Copiar y adaptar

Biblioteca de métodos y plantillas reutilizables.

03

Corregir y ampliar

Métodos paso a paso, errores y ejercicios tipo examen.

04

Repasar

Chuleta ultrarrápida y checklist de los 2 minutos.

Índice

27 TEMAS · 5 PARTES

PARTE I	Fundamentos del lenguaje	3
01	Variables y tipos	3
02	Operadores, números y conversiones	4
03	Entrada y salida por consola	5
04	Condicionales: if y when	5
05	Bucles	6
06	Funciones	7
07	Strings: trabajar con texto	8
PARTE II	Colecciones y estructuras de datos	9
08	Arrays	9
09	Listas: el tema estrella	10
10	Lambdas e <code>it</code>	10
11	Buscar, filtrar y transformar	11
12	Lo mismo, a mano con <code>for</code>	13
13	Matrices	13
14	Maps y Sets	15
PARTE III	Objetos, null y Android	16
15	Null safety	16
16	Clases y data class	17
17	Patrón estrella: lista de objetos	18
18	Excepciones y tipos de error	18
19	Android / Jetpack Compose	19
PARTE IV	Modo examen	21
20	Plantilla completa: lista de objetos	21
21	¿Qué me piden? → Qué uso	22
22	Método para CORREGIR código	23
23	Método para AÑADIR una funcionalidad	23
24	Errores que debes reconocer	24
25	Ejercicios tipo examen	25
PARTE V	Repaso final	27
26	Chuleta ultrarrápida	27
27	Checklist final y plan de estudio	28

Cómo usar esta guía

LÉELA UNA VEZ · CONSÚLTALA SIEMPRE

OBJETIVO DEL EXAMEN

Coger un código que no has escrito tú, entenderlo, localizar sus errores y añadirle una funcionalidad nueva.

1 Lee el código en voz alta

Recorre el programa línea a línea y di qué hace cada una. Si puedes explicarlo, puedes modificarlo.

2 Copia con cabeza

La biblioteca está hecha para copiar y **adaptar**, no para copiar a ciegas. Si tu lista se llama `alumnos` y el método usa `personas`, cámbialo.

3 Antes de pegar, 4 comprobaciones

① Nombre de la lista · ② tipo de objeto · ③ nombres de las propiedades · ④ tipo que devuelve la función.

Leyenda

Título del bloque de código

PLANTILLA

```
val x = 5 // ✓ verde: correcto
x = 6    // ✗ rojo: error
```

→ Explicación: qué ocurre al ejecutar el código.

TRUCO DE EXAMEN

Atajos y patrones que suelen caer.

ERROR TÍPICO

Fallos que el profesor pondrá para que los corrijas.

La etiqueta **PLANTILLA** marca código pensado para copiar y adaptar tal cual.

Fundamentos del lenguaje

Las piezas con las que se construye cualquier programa: guardar datos, operar con ellos, tomar decisiones, repetir y organizar el código en funciones.

val · var

tipos

operadores

readln

if · when

for · while

fun

String

01 Variables y tipos

val · var · Int · Double · String · Char · Boolean

Una variable es un espacio con nombre donde guardamos un dato. En Kotlin se declaran con `val` (no se puede reasignar) o `var` (se puede cambiar).

val — no reassignable

```
val nombre = "Ana"
println(nombre)
nombre = "Luis" // X Val cannot be reassigned
```

→ `val` crea una referencia que no puedes volver a asignar. Úsalo por defecto.

var — reassignable

```
var edad = 20
println(edad) // 20
edad = 21 // cambiamos el valor
println(edad) // 21
```

→ Usa `var` cuando el programa necesita cambiar el valor: contadores, acumuladores, estados...

Tipos básicos

```
val edad: Int = 20
val precio: Double = 4.99
val nombre: String = "Ana"
val inicial: Char = 'A'
val aprobado: Boolean = true
val grande: Long = 3_000_000_000L
```

→ Tras los dos puntos va el tipo. Kotlin suele deducirlo (inferencia): `val edad = 20` ya es `Int`.

val no significa «inmutable»

```
val lista = mutableListOf(1, 2)
lista.add(3) // ✓ el contenido sí
lista = mutableListOf(9) // X la referencia no
```

→ `val` impide **reasignar** la variable, no modificar el objeto al que apunta.



TRUCO DE EXAMEN

Si una variable no necesita cambiar → `val`. Si el programa la modifica → `var`. Ante la duda empieza con `val`: el compilador te avisará si hace falta cambiarla.



ERROR TÍPICO

No confundas `Char` (`'A'`) con `String` (`"A"`). Comillas simples = un carácter; comillas dobles = texto.

Aritmética

```
val a = 10
val b = 3
val suma = a + b           // 13
val resta = a - b           // 7
val multiplicacion = a * b  // 30
val division = a / b        // 3 ¡entera!
val resto = a % b           // 1
val real = a / 3.0          // 3.333...
val real2 = a.toDouble() / b // 3.333...
```

→ `Int / Int` da `Int` : se pierden los decimales. Si uno de los dos es `Double` , el resultado es `Double` .

Condiciones combinadas

```
val edad = 20
val tieneEntrada = true
if (edad >= 18 && tieneEntrada) {
    println("Puede entrar")
}
val dia = 6
val esFinde = dia == 6 || dia == 7 // true
val laborable = !esFinde           // false
```

→ `&&` exige que ambas sean verdaderas · con `||` basta una · `!` niega.

Números: biblioteca

PLANTILLA

```
import kotlin.math.* // arriba del fichero

val esPar = numero % 2 == 0
val esMultiplo3 = numero % 3 == 0
val mayor = maxOf(a, b, c)
val menor = minOf(a, b, c)
val positivo = abs(-7) // 7
val cuadrado = 3.0.pow(2) // 9.0
val raiz = sqrt(16.0) // 4.0
val redondo = 4.6.roundToInt() // 5
println("%.2f".format(4.5)) // 4.50
```

→ Los `import` van siempre al principio del fichero, nunca dentro de una función.

Asignación compuesta e incremento

```
var total = 10
total += 5 // 15 (total = total + 5)
total -= 3 // 12
total *= 2 // 24
total++ // 25
total-- // 24
```

Comparar

```
val edad = 20
println(edad >= 18) // true
println(edad == 20) // true
println(edad != 15) // true
```

→ `==` compara valores; `=` asigna. Con Strings, `==` compara el contenido.

Conversiones

```
val numero = "25".toInt() // String → Int
val decimal = "3.5".toDouble() // String → Double
val texto = numero.toString() // Int → String
val entero = 7.9.toInt() // 7 (trunca)
val seguro = "abc".toIntOrNull() // null
```

→ Kotlin no trata `"25"` como número automáticamente: hay que convertir.

**ERROR TÍPICO**

`"Hola".toInt()` lanza `NumberFormatException` porque no es un número válido. Si el dato viene del usuario, usa `toIntOrNull()` y decide qué hacer con el `null` .

Mostrar por pantalla

```
println("Hola")           // escribe y salta de línea
print("Sin salto ")       // escribe sin saltar
println()                 // línea en blanco
```

Lectura segura de números

```
print("Edad: ")
val edad = readln().toIntOrNull()
if (edad == null) {
    println("Eso no es un número")
} else {
    println("El año que viene: ${edad + 1}")
}
```

Leer del teclado

```
print("Nombre: ")
val nombre = readln()

print("Edad: ")
val edad = readln().toInt()
```

→ `readln()` devuelve siempre un `String` : para números hay que convertir. En versiones antiguas verás `readLine()!!`.

Menú típico de examen

PLANTILLA

```
var opcion: Int
do {
    println("1. Añadir 2. Mostrar 0. Salir")
    opcion = readln().toIntOrNull() ?: -1
    when (opcion) {
        1 -> agregar()
        2 -> mostrar()
        0 -> println("Adiós")
        else -> println("Opción no válida")
    }
} while (opcion != 0)
```

→ `do-while` muestra el menú al menos una vez; `when` reparte cada opción.

04 Condicionales: if y when

if / else

```
val edad = 17
if (edad >= 18) {
    println("Es mayor de edad")
} else {
    println("Es menor de edad")
}
```

→ Primero se evalúa `edad >= 18` : si es `true` entra en el primer bloque; si es `false`, en `else`.

if como expresión

```
val mayor = if (a > b) a else b
val estado = if (nota >= 5) "Apto" else "No apto"
```

→ En Kotlin `if` devuelve un valor. No existe el ternario `c ? a : b` de Java.

else if

```
val nota = 7
if (nota >= 9) {
    println("Sobresaliente")
} else if (nota >= 5) {
    println("Aprobado")
} else {
    println("Suspenso")
}
```

→ Se revisan de arriba abajo: la primera que se cumple se ejecuta y el resto se ignora. **El orden importa**: si pusieras primero `nota >= 5`, un 10 diría «Aprobado».

when — varias opciones

```
val opcion = 2
when (opcion) {
    1 -> println("Añadir")
    2 -> println("Eliminar")
    3 -> println("Mostrar")
    else -> println("Opción incorrecta")
}
```

→ Muy cómodo con varias opciones concretas. En un examen suele aparecer como menú.

when avanzado

```
when (dia) {
    1, 2, 3, 4, 5 -> println("Laborable")
    6, 7 -> println("Fin de semana")
    else -> println("Día no válido")
}

when (nota) {
    10 -> println("Matrícula")
    in 5..9 -> println("Aprobado")
    else -> println("Suspendido")
}
```

when sin argumento (devuelve valor)

```
val calificacion = when {
    nota >= 9 -> "Sobresaliente"
    nota >= 7 -> "Notable"
    nota >= 5 -> "Aprobado"
    else -> "Suspendido"
}
```

→ Sin argumento, cada rama es una condición libre (útil con `Double`). Si se usa como valor, el `else` es obligatorio.



TRUCO DE EXAMEN

Si ves un menú con 1, 2, 3..., piensa inmediatamente en `when`.

05 Bucles: repetir código

for · rangos · while · do-while

RANGO	VALORES	ÚSALO PARA
1..5	1 2 3 4 5	Contar de 1 a N. Incluye el final.
0 until 5 · 0.. 5	0 1 2 3 4	Índices: excluye el final.
5 downTo 1	5 4 3 2 1	Cuentas atrás.
0..10 step 2	0 2 4 6 8 10	Ir de N en N.
lista.indices	0 ... size-1	Recorrer posiciones de forma segura.

for con rango

```
for (i in 1..5) {
    println(i) // 1 2 3 4 5
}

for (i in 0 until 5) {
    println(i) // 0 1 2 3 4
}
```

→ `..` incluye el final; `until` (o `..
<`) lo excluye, perfecto para índices.

downTo y step

```
for (i in 10 downTo 1) print("$i ") // 10 ... 1
for (i in 0..10 step 2) print("$i ") // 0 2 ... 10
for (i in 9 downTo 0 step 3) {
    print("$i ") // 9 6 3 0
}
```

◀ Recorrer con índice

```
for (i in nombres.indices) {
    println("Posición $i: ${nombres[i]}")
}
for ((i, nombre) in nombres.withIndex()) {
    println("$i → $nombre")
}
```

→ `indices` da las posiciones válidas (0, 1, 2 para 3 elementos). `withIndex()` da posición y valor a la vez.

◀ break, continue y repeat

```
for (n in numeros) {
    if (n < 0) continue // siguiente vuelta
    if (n == 0) break // sale del bucle
    println(n)
}
repeat(3) { println("Hola") } // 3 veces
```

◀ Recorrer una lista

```
val nombres = listOf("Ana", "Luis", "Marta")
for (nombre in nombres) {
    println(nombre)
}
```

→ En cada vuelta, `nombre` es directamente un elemento de la lista. No necesitas índices.

◀ while y do-while

```
var intentos = 0
while (intentos < 3) {
    println("Intento ${intentos + 1}")
    intentos++
}

do {
    print("Número positivo: ")
    val n = readln().toIntOrNull() ?: 0
} while (n <= 0)
```

→ `while` comprueba antes (puede no ejecutarse nunca). `do-while` se ejecuta **al menos una vez**.

"Ana"

0

"Luis"

1

"Marta"

2

X

3

`size = 3`, pero las posiciones válidas son **0, 1 y 2**. `lista[3]` no existe → `IndexOutOfBoundsException`.



ERROR TÍPICO

Aparece muchísimo: `for (i in 0..lista.size)`. Si `size` vale 3, intentará acceder también a `lista[3]`. Usa `lista.indices` o `0` until `lista.size`.

06 Funciones: dividir el programa en tareas

fun • parámetros • return

```
fun sumar(a: Int, b: Int): Int { return a + b }
```

PALABRA CLAVE

Declara una función.

NOMBRE

Un verbo que describa la tarea.

PARÁMETROS

Datos que recibe, cada uno con su tipo.

TIPO DE RETORNO

Lo que devuelve. Si no devuelve nada, se omite.

CUERPO

El código; `return` entrega el resultado.

◀ Función con retorno

```
fun sumar(a: Int, b: Int): Int {
    return a + b
}
val resultado = sumar(5, 3)
println(resultado) // 8
```

→ Al llamar `sumar(5, 3)`: `a` recibe 5, `b` recibe 3 y `return` devuelve 8.

◀ Función sin retorno

```
fun mostrarNombre(nombre: String) {
    println("Nombre: $nombre")
}
mostrarNombre("Ana")
```

→ No devuelve un valor útil, por eso no hay `: Tipo` tras los paréntesis (implícitamente `Unit`).

Función de una línea

```
fun doble(numero: Int): Int = numero * 2
fun esPar(n: Int) = n % 2 == 0
```

→ Forma abreviada cuando el cuerpo es una sola expresión.

Parámetros por defecto y con nombre

```
fun saludar(nombre: String = "invitado") {
    println("Hola, $nombre")
}
saludar()           // Hola, invitado
saludar("Ana")      // Hola, Ana
val p = Persona(nombre = "Ana", edad = 20)
```



TRUCO DE EXAMEN

Cuando el ejercicio pide añadir una funcionalidad, casi siempre la mejor solución es crear una **función separada** que haga esa tarea y llamarla desde donde toque.



ERROR TÍPICO

Si declaras `: Int`, todos los caminos deben terminar en `return`. Si no: «A 'return' expression required in a function with a block body».

07 Strings: trabajar con texto

length · interpolación · métodos

Lo básico

```
val nombre = "Samara"
println(nombre.length)           // 6
println(nombre.toUpperCase())    // SAMARA
println(nombre.toLowerCase())    // samara
println(nombre[0])               // S (Char)
println(nombre.contains("mar"))  // true
```

→ El primer carácter está en la posición 0 y el último en `length - 1`.

Interpolación

```
val nombre = "Ana"
val edad = 20
println("Me llamo $nombre")
println("Tengo $edad años")
println("El año que viene tendré ${edad + 1}")
println("Nombre: ${persona.nombre}")
```

→ `$variable` mete el valor en el texto. `${...}` permite expresiones y propiedades.

Operaciones frecuentes

PLANTILLA

```
val limpio = texto.trim()        // quita espacios
val cambiado = texto.replace("Hola", "Adiós")
val palabras = texto.split(" ") // List<String>
val invertido = texto.reversed()
val trozo = texto.substring(0, 4) // pos. 0..3
val pos = texto.indexOf("a")     // -1 si no está
val empieza = texto.startsWith("Ho")
val acaba = texto.endsWith("!")
```

Comparar y comprobar

```
if (texto.equals("hola", ignoreCase = true)) {
    println("Iguales sin mirar mayúsculas")
}
if (texto.isBlank()) println("Vacío") // "" o " "
if (texto.isEmpty()) println("0 caracteres")
```

Recorrer caracteres

```
var vocales = 0
for (c in texto.lowercase()) {
    if (c in "aeiou") vocales++
}
val digitos = texto.count { it.isDigit() }
val primera = texto.first() // Char
```



ERROR TÍPICO

`"$persona.nombre"` imprime el objeto entero seguido de `.nombre`. Con propiedades usa siempre llaves: `"${persona.nombre}"`.



TRUCO DE EXAMEN

Cuando te pidan manipular texto, revisa: `length`, `contains`, `replace`, `split`, `uppercase`, `lowercase` y `trim`.

Colecciones y estructuras de datos

Guardar muchos datos juntos y trabajar con ellos: el bloque que más aparece en los exámenes. Incluye la biblioteca de operaciones lista para copiar.

Array

List · MutableList

lambdas · it

find · filter · map

matrices

Map · Set

08 Arrays: tamaño fijo

arrayOf · IntArray · size · indices

Un array guarda varios valores y cada uno tiene una posición. Lo importante es dominar los **índices**: un array de 3 elementos tiene las posiciones 0, 1 y 2.

Crear y acceder

```
val numeros = arrayOf(10, 20, 30)
println(numeros[0]) // 10
println(numeros[2]) // 30
numeros[0] = 99 // cambia el primero

val enteros = IntArrayOf(1, 2, 3)
val ceros = IntArray(5) // [0, 0, 0, 0, 0]
val textos = Array(3) { "" } // 3 textos vacíos
```

size e indices

```
println(numeros.size) // 3
for (i in numeros.indices) {
    println("Índice $i: ${numeros[i]}")
}
for (numero in numeros) println(numero)
```

→ `size` dice cuántos elementos hay; `indices`, qué posiciones son válidas.

Buscar un valor

```
var encontrado = false
for (numero in numeros) {
    if (numero == 20) {
        encontrado = true
        break
    }
}
// atajo: val encontrado = 20 in numeros
```

Sumar y media

```
var suma = 0
for (numero in numeros) {
    suma += numero
}
val media = suma.toDouble() / numeros.size
// atajos: numeros.sum(), numeros.average()
```

Array ≠ MutableList

```
val array = arrayOf(1, 2, 3)
array.add(4) // X Array no tiene add
val lista = mutableListOf(1, 2, 3)
lista.add(4) // ✓ sí se puede
```

→ Un `Array` tiene tamaño fijo. Una `MutableList` puede crecer y reducirse.

Máximo y mínimo

```
var maximo = numeros[0]
var minimo = numeros[0]
for (numero in numeros) {
    if (numero > maximo) maximo = numero
    if (numero < minimo) minimo = numero
}
// atajos: numeros.maxOrNull(), numeros.minOrNull()
```

→ Empieza con el primer elemento, no con 0: si todos fueran negativos, 0 daría un máximo falso.



ERROR TÍPICO

`println(numeros)` muestra algo como `[Ljava.lang.Integer;@1b6d3586`. Para ver el contenido usa `numeros.contentToString()` o `numeros.joinToString()`.

List — solo lectura

```
val nombres = listOf("Ana", "Luis", "Marta")
println(nombres[0])    // Ana
println(nombres.size)  // 3
nombres.add("Pedro")   // X Unresolved reference
```

→ `listOf` crea una lista que puedes consultar pero no modificar con `add` / `remove`.

Modificar una a una

PLANTILLA

```
nombres.add("Lucía")    // al final
nombres.add(1, "Carlos") // en la posición 1
nombres.addAll(otraLista) // añade varios
nombres.remove("Ana")   // elimina ese VALOR
nombres.removeAt(0)     // elimina la POSICIÓN 0
nombres[0] = "Mario"    // cambia un elemento
nombres.clear()         // la vacía entera
```

→ `remove` trabaja con el **elemento**; `removeAt` trabaja con la **posición**.

Ordenar: sorted vs sort

```
val numeros = mutableListOf(3, 1, 2)
val copia = numeros.sorted() // [1, 2, 3] nueva
println(numeros)             // [3, 1, 2] intacta
numeros.sort()               // ordena la MISMA
numeros.sortDescending()     // [3, 2, 1]
personas.sortBy { it.edad }  // objetos, en sitio
```

→ `sorted...` devuelve una lista nueva; `sort...` modifica la original (solo `MutableList`).

MutableList — modificable

```
val nombres = mutableListOf("Ana", "Luis")
nombres.add("Marta")
println(nombres)    // [Ana, Luis, Marta]

// vacía: hay que indicar el tipo
val vacia = mutableListOf<String>()
```

→ Si la lista empieza vacía, Kotlin no puede deducir el tipo: escríbelo entre `< >`.

Consultar

PLANTILLA

```
nombres.size           // n° de elementos
nombres.contains("Mario") // true / false
"Mario" in nombres     // lo mismo
nombres.indexOf("Mario") // posición o -1
nombres.isEmpty()      // ¿está vacía?
nombres.isNotEmpty()   // 
nombres.first()         // error si vacía
nombres.firstOrNull()   // null si vacía
```

Lista de objetos

```
val personas = mutableListOf<Persona>()
personas.add(Persona("Ana", 20))
personas.addAll(
    listOf(
        Persona("Luis", 17),
        Persona("Marta", 25)
    )
)
personas[0] = Persona("Ana", 21) // reemplazar
```

ERROR TÍPICO

Si el enunciado dice «elimina el elemento de la posición 2», necesitas `removeAt(2)`, no `remove(2)`. En una `MutableList<Int>`, `remove(2)` borraría el **valor** 2.

10 Lambdas e `it` explicadas

{ elemento -> resultado }

Cuando ves `{ it.edad >= 18 }` no es magia: es una **función pequeña** que recibe un elemento y devuelve un resultado (aquí `true` o `false`).

```
personas.filter { it.edad >= 18 }
```

COLECCIÓN

La lista que se recorre.

OPERACIÓN

Qué hacer con los resultados.

LAMBDA { }

Se ejecuta una vez por cada elemento.

IT

El elemento de la vuelta actual.

CONDICIÓN`true` → se queda · `false` → fuera.

◀ Parámetro con nombre vs it

```
val aprobados = alumnos.filter { alumno ->
    alumno.nota >= 5
}
// lo mismo, con el nombre implícito it
val aprobados2 = alumnos.filter { it.nota >= 5 }
```

→ Antes de `->` va el nombre del parámetro; después, lo que se devuelve (la última expresión). Si hay un solo parámetro y no lo nombras, se llama `it`.

◀ Ejemplo con map

```
val nombres = alumnos.map { alumno ->
    alumno.nombre
}
```

→ Por cada `Alumno` obtenemos su nombre: el resultado pasa de `List<Alumno>` a `List<String>`.

◀ Ejemplo con forEach

```
alumnos.forEach { alumno ->
    println(alumno.nombre)
}
alumnos.forEachIndexed { i, a ->
    println("$i: ${a.nombre}")
}
```

→ `forEach` ejecuta el bloque por cada elemento. No crea una lista nueva.

11 Buscar, filtrar y transformar

biblioteca de operaciones con colecciones

En todos los ejemplos: `data class Persona(val nombre: String, var edad: Int)` y `val personas = mutableListOf<Persona>()`. **Cambia los nombres por los de tu ejercicio.**

FUNCIÓN	DEVUELVE	SI NINGUNO CUMPLE...
<code>find { } · firstOrNull { }</code>	el primer elemento que cumple (<code>T?</code>)	<code>null</code>
<code>indexOfFirst { }</code>	su posición (<code>Int</code>)	<code>-1</code>
<code>filter { }</code>	una <code>List</code> con los que cumplen	lista vacía
<code>map { }</code>	una <code>List</code> transformada (otro tipo)	— (mismo tamaño que la original)
<code>any · all · none { }</code>	<code>Boolean</code>	lista vacía: <code>any</code> → <code>false</code> · <code>all</code> → <code>true</code> · <code>none</code> → <code>true</code>
<code>count { }</code>	cuántos cumplen (<code>Int</code>)	<code>0</code>
<code>sumOf { }</code>	la suma de lo que devuelve la lambda	<code>0</code>
<code>maxByOrNull · minByOrNull</code>	el elemento con el mayor / menor valor (<code>T?</code>)	<code>null</code> (lista vacía)
<code>sortedBy · sortedByDescending</code>	una <code>List</code> nueva ordenada	—
<code>forEach { }</code>	nada (<code>Unit</code>): solo ejecuta	—

◀ find — encontrar uno

PLANTILLA

```
val persona = personas.find {
    it.nombre == "Ana"
}
```

→ Recorre la lista hasta el primer elemento que cumple. Si no hay ninguno, devuelve `null`.

◀ indexOfFirst — su posición

```
val indice = personas.indexOfFirst {
    it.nombre == "Ana"
}
if (indice != -1) {
    personas[indice] = Persona("Ana", 21)
}
```

filter — seleccionar varios

```
val mayores = personas.filter {  
    it.edad >= 18  
}
```

→ Devuelve una **nueva lista** con los que cumplen. La original no cambia.

count — contar

```
val cantidad = personas.count { it.edad >= 18 }
```

Sumas, medias, máximos

```
val total = personas.sumOf { it.edad }  
val media = personas.map { it.edad }.average()  
val mayor = personas.maxByOrNull { it.edad }  
val menor = personas.minByOrNull { it.edad }  
// mayor y menor son Persona? (null si vacía)
```

Eliminar con condición

```
personas.removeIf { it.edad < 18 }  
personas.removeAll { it.edad < 18 } // Kotlin puro
```

→ Eliminan **todos** los elementos que cumplen la condición.

Buscar y comprobar

PLANTILLA

```
val persona = personas.find {  
    it.nombre == nombreBuscado  
}  
if (persona != null) {  
    println("Encontrada: ${persona.nombre}")  
} else {  
    println("No encontrada")  
}
```

map — transformar

```
// List<Persona> → List<String>  
val nombres = personas.map { it.nombre }  
// List<Persona> → List<Int>  
val edades = personas.map { it.edad }
```

any / all / none

```
val hayAdultos = personas.any { it.edad >= 18 }  
val todosAdultos = personas.all { it.edad >= 18 }  
val ningunMenor = personas.none { it.edad < 18 }
```

→ **any** : ¿existe al menos uno? · **all** : ¿cumplen todos? · **none** : ¿no cumple ninguno?

Ordenar

```
val porEdad = personas.sortedBy { it.edad }  
val desc = personas.sortedByDescending { it.edad }  
val porNombre = personas.sortedBy { it.nombre }
```

Encadenar operaciones

```
val nombresAdultos = personas  
    .filter { it.edad >= 18 }  
    .sortedBy { it.nombre }  
    .map { it.nombre }  
println(nombresAdultos.joinToString(", "))
```

→ Cada paso recibe el resultado del anterior. `joinToString` une la lista en un único texto.



TRUCO DE EXAMEN — MEMORIZA ESTA FRASE

`filter` = seleccionar · `map` = transformar · `find` = encontrar uno · `any` = ¿existe alguno? · `all` = ¿todos? · `count` = ¿cuántos?

12 Lo mismo, a mano con `for`

cuando no puedes usar `filter/find...`

Si el profesor pide hacerlo «a mano», o para entender qué hacen por dentro. Todas siguen el **patrón acumulador**: variable *antes* del bucle → se actualiza *dentro* → se usa *después*.

Buscar (= `find`)

```
fun buscarPersona(nombre: String): Persona? {
    for (persona in personas) {
        if (persona.nombre == nombre) {
            return persona
        }
    }
    return null
}
```

Filtrar (= `filter`)

```
fun obtenerMayores(): List<Persona> {
    val resultado = mutableListOf<Persona>()
    for (persona in personas) {
        if (persona.edad >= 18) {
            resultado.add(persona)
        }
    }
    return resultado
}
```

Contar (= `count`)

```
fun contarMayores(): Int {
    var contador = 0
    for (persona in personas) {
        if (persona.edad >= 18) {
            contador++
        }
    }
    return contador
}
```

Sumar (= `sumOf`)

```
fun sumarEdades(): Int {
    var suma = 0
    for (persona in personas) {
        suma += persona.edad
    }
    return suma
}
```

Posición (= `indexOfFirst`)

```
fun posicionDe(nombre: String): Int {
    for (i in personas.indices) {
        if (personas[i].nombre == nombre) {
            return i
        }
    }
    return -1
}
```

El mayor (= `maxByOrNull`)

```
fun personaMayor(): Persona? {
    if (personas.isEmpty()) return null
    var mayor = personas[0]
    for (persona in personas) {
        if (persona.edad > mayor.edad) {
            mayor = persona
        }
    }
    return mayor
}
```

13 Matrices: filas y columnas

`Array(3) { IntArray(3) }`

Una matriz es una estructura de dos dimensiones: piensa en ella como una **tabla**. Se accede con `matriz[filas][columna]`.

	col 0	col 1	col 2
fila 0	<code>[0][0]</code>	<code>[0][1]</code>	<code>[0][2]</code>
fila 1	<code>[1][0]</code>	<code>[1][1]</code>	<code>[1][2]</code>
fila 2	<code>[2][0]</code>	<code>[2][1]</code>	<code>[2][2]</code>

Fila 1 = `matriz[1]` (un `IntArray` completo).

Columna 2 = el elemento 2 de cada fila.

`matriz[1][2]` → primero la fila, luego la columna.

`matriz.size` = nº de filas · `matriz[0].size` = nº de columnas.

En una 3×3 la última posición es `[2][2]`.

Crear

```
val matriz = Array(3) { IntArray(3) } // 3x3 a 0
matriz[0][0] = 10
matriz[1][2] = 20
println(matriz[1][2]) // 20

val m = arrayOf( // con valores
    intArrayOf(1, 2, 3),
    intArrayOf(4, 5, 6)
)
```

→ `Array(3)` crea 3 filas; cada fila es un `IntArray(3)`, es decir, 3 columnas.

Sumar todos los valores

```
var suma = 0
for (fila in matriz.indices) {
    for (columna in matriz[fila].indices) {
        suma += matriz[fila][columna]
    }
}
println(suma)
```

Sumar una columna

```
val columna = 0
var suma = 0
for (fila in matriz.indices) {
    suma += matriz[fila][columna]
}
```

→ Columna fija, se mueve la fila.

Diagonal principal

```
var sumaDiagonal = 0
for (i in matriz.indices) {
    sumaDiagonal += matriz[i][i] // [0][0] [1][1]...
}
```

Rellenar

```
for (fila in matriz.indices) {
    for (columna in matriz[fila].indices) {
        print("Valor [$fila][$columna]: ")
        matriz[fila][columna] = readln().toInt()
    }
}
```

Recorrer

PLANTILLA

```
for (fila in matriz.indices) {
    for (columna in matriz[fila].indices) {
        println(matriz[fila][columna])
    }
}
```

→ El `for` exterior recorre filas; para cada fila, el interior recorre sus columnas. Por eso hay dos bucles.

Imprimir como tabla

```
for (fila in matriz) {
    for (valor in fila) {
        print("$valor\t")
    }
    println() // salto al acabar cada fila
}
```

Sumar una fila

```
val fila = 0
var suma = 0
for (columna in matriz[fila].indices) {
    suma += matriz[fila][columna]
}
```

→ Fila fija, se mueve la columna.

Buscar un valor

```
var encontrado = false
for (fila in matriz.indices) {
    for (columna in matriz[fila].indices) {
        if (matriz[fila][columna] == 10) {
            encontrado = true
            println("Está en [$fila][$columna]")
        }
    }
}
```



ERROR TÍPICO

En una matriz 3x3, `[3][3]` es incorrecto: la última posición es `[2][2]`. Y no confundas filas con columnas: `matriz.size` son filas; `matriz[0].size`, columnas.

Map : clave → valor

```
val stock = mutableMapOf("Pan" to 10, "Leche" to 5)
stock["Queso"] = 3 // añadir o cambiar
println(stock["Pan"]) // 10 (Int?)
println(stock.getOrElse("Huevos", 0))
stock.remove("Leche")
if ("Pan" in stock) println("Hay pan")
for ((producto, cantidad) in stock) {
    println("$producto: $cantidad")
}
```

→ Cada clave es única. `stock["X"]` devuelve `null` si la clave no existe.

Contar apariciones

PLANTILLA

```
val conteo = mutableMapOf<String, Int>()
for (palabra in palabras) {
    conteo[palabra] = (conteo[palabra] ?: 0) + 1
}
```

Set : sin repetidos

```
val unicos = mutableSetOf(1, 2, 2, 3)
// unicos = [1, 2, 3]
unicos.add(3) // no se repite
val sinRepes = lista.distinct() // List
```

¿Qué estructura elijo?

ESTRUCTURA	CREAR	¿CRECE?	ÚSALA CUANDO...
Array	<code>arrayOf(1, 2, 3)</code>	No	El tamaño es fijo y conocido (7 días, 10 notas...).
List	<code>listOf("A", "B")</code>	No (solo lectura)	Solo vas a consultar los datos.
MutableList	<code>mutableListOf<T>()</code>	Sí	Hay que añadir, eliminar o modificar. La más habitual en el examen.
Matriz	<code>Array(3) { IntArray(3) }</code>	No	Datos en forma de tabla: filas × columnas (tableros, notas por evaluación...).
MutableMap	<code>mutableMapOf("a" to 1)</code>	Sí	Buscas por una clave: código → producto, nombre → nota.
MutableSet	<code>mutableSetOf(1, 2)</code>	Sí	No quieres elementos repetidos.



TRUCO DE EXAMEN

«añadir / eliminar» → `MutableList` · «tamaño fijo» → `Array` · «tabla» → matriz · «buscar por código» → `Map` · «sin repetidos» → `Set`.



ERROR TÍPICO

Declarar la lista con `listOf` y luego intentar `add` o `remove`. Si el programa modifica la colección, tiene que ser `mutableListOf` (o `mutableMapOf` / `mutableSetOf`).

Objetos, null y Android

Cómo modelar datos con clases, cómo evitar los fallos por `null`, cómo distinguir los tipos de error y cómo leer y modificar una app de Jetpack Compose.

String?

?. · ?: · !!

class · data class

copy

try / catch

@Composable

remember

15 Null safety: imprescindible para corregir

String vs String?

En Kotlin, `String` y `String?` no son lo mismo: `String` nunca puede ser `null`; `String?` sí. El compilador te obliga a tratar ese posible `null`.

Tipos nullables

```
var nombre: String = "Ana"
nombre = null           // X no permitido
var apellido: String? = null
apellido = "García"     // ✓ permitido
```

?: — operador Elvis

```
val texto = apellido ?: "Sin apellido"
val longitud = apellido?.length ?: 0
```

→ Si hay valor, se usa; si es `null`, se usa el de la derecha.

if (x != null) — smart cast

```
if (apellido != null) {
    println(apellido.length) // aquí ya es String
}
```

→ Dentro del `if`, Kotlin sabe que no es nulo y te deja usarlo directamente.

?. — llamada segura

```
var apellido: String? = null
println(apellido?.length) // null (no falla)
```

→ Si `apellido` es `null`, Kotlin no intenta acceder a `length` y devuelve `null`.

!! — «confía en mí»

```
val longitud = apellido!!.length // X NPE si null
```

→ Obliga a tratar el valor como no nulo. Si realmente es `null`, lanza `NullPointerException`.

let

```
apellido?.let {
    println("Apellido: $it") // si no es null
}
```

find + null

PLANTILLA

```
val persona = personas.find {
    it.nombre == buscado
}
if (persona != null) {
    println(persona.edad)
}
// o, en una sola línea:
println(persona?.edad ?: "No existe")
```


QUIERO...	USO
Seguir aunque sea null (el resultado será null)	<code>x?.propiedad</code>
Un valor por defecto si es null	<code>x ?: valorPorDefecto</code>
Hacer varias cosas solo si no es null	<code>if (x != null) { ... } · x?.let { ... }</code>
Afirmar que nunca será null (arriesgado)	<code>x!!</code>



ERROR TÍPICO

En un examen, si puedes resolverlo con `?.` o `?:`, es una opción mucho más segura que `!!`.

16 Clases y data class

el molde y los objetos

class

```
class Persona(
    val nombre: String,
    var edad: Int
)
val persona = Persona("Ana", 20)
println(persona.nombre)
persona.edad = 21 // ✓ es var
persona.nombre = "Eva" // ✗ es val
```

→ La clase define qué datos tiene una Persona (el molde). Cada `Persona("Ana", 20)` crea un **objeto**.

Lista de objetos

```
val productos = mutableListOf(
    Producto("Pan", 1.50),
    Producto("Leche", 1.20),
    Producto("Queso", 4.00)
)
for (p in productos) {
    println("${p.nombre}: ${p.precio} €")
}
```

→ Cada elemento es un `Producto`, por eso dentro del bucle accedes a `p.nombre` y `p.precio`.

Herencia y toString

```
open class Animal(val nombre: String) {
    open fun sonido() = "..."
}
class Perro(nombre: String) : Animal(nombre) {
    override fun sonido() = "Guau"
    override fun toString() = "Perro $nombre"
}
```

→ Las clases son `final` por defecto: para heredar, `open`; para redefinir un método, `open + override`.

data class

```
data class Producto(
    val nombre: String,
    val precio: Double
)
val pan = Producto("Pan", 1.50)
println(pan) // Producto(nombre=Pan, precio=1.5)
```

→ Genera automáticamente `toString()`, `equals()` (compara contenido) y `copy()`. Ideal para listas de objetos.

copy

```
val masCaro = pan.copy(precio = 1.80)
// Producto(nombre=Pan, precio=1.8)
```

→ Crea un objeto nuevo igual al original salvo lo que cambies.

Clase con métodos

```
class Cuenta(
    val titular: String,
    var saldo: Double
) {
    fun ingresar(cantidad: Double) {
        if (cantidad > 0) saldo += cantidad
    }
    fun retirar(cantidad: Double): Boolean {
        if (cantidad > saldo) return false
        saldo -= cantidad
        return true
    }
}
val cuenta = Cuenta("Ana", 100.0)
cuenta.ingresar(50.0) // saldo = 150.0
```

17 Patrón estrella: modificar una lista de objetos

data class + MutableList

Este patrón es **muy probable** en el examen: una `data class`, una `MutableList` y operaciones de añadir, filtrar, buscar, modificar y eliminar.

Datos de partida

```
data class Alumno(  
    val nombre: String,  
    var nota: Double  
)  
  
val alumnos = mutableListOf(  
    Alumno("Ana", 8.5),  
    Alumno("Luis", 4.5)  
)  
  
alumnos.add(Alumno("Marta", 7.0))
```

→ Como la lista es `MutableList`, podemos añadir otro `Alumno`.

Modificar una propiedad

```
val alumno = alumnos.find { it.nombre == "Luis" }  
if (alumno != null) {  
    alumno.nota = 5.0 // requiere var nota  
}
```

Filtrar aprobados

```
val aprobados = alumnos.filter {  
    it.nota >= 5  
}
```

→ `it` representa al `Alumno` que se comprueba en cada vuelta.

Buscar por nombre

```
val alumno = alumnos.find {  
    it.nombre == "Luis"  
}
```

→ Puede ser `null` si no existe Luis: por eso `find` va de la mano de null safety.

Eliminar

```
alumnos.removeIf {  
    it.nombre == "Luis"  
}
```

→ `removeIf` elimina **todos** los elementos que cumplan la condición.



TRUCO DE EXAMEN

`data class` + `MutableList` + `filter` / `find` / `removeIf`. Si dominas estas cinco piezas, tienes resuelto el ejercicio más habitual.

18 Excepciones y tipos de error

try · catch · compilación vs ejecución

TIPO 1

De compilación

El programa ni siquiera se construye.
El IDE lo subraya en rojo.

`Type mismatch`
`Unresolved reference`
`Val cannot be reassigned`

TIPO 2

De ejecución

Compila, pero falla mientras funciona y el programa se detiene.

`IndexOutOfBoundsException`
`NullPointerException`
`NumberFormatException`

TIPO 3

Lógico

Funciona sin fallar, pero el resultado es incorrecto.

> en vez de ≥
orden de los else if
división entera

try / catch

```
try {
    val numero = "abc".toInt()
    println(numero)
} catch (e: NumberFormatException) {
    println("El texto no es un número")
} finally {
    println("Esto se ejecuta siempre")
}
```

→ Se intenta ejecutar lo de `try`. Si ocurre esa excepción, se salta a `catch`.

try como expresión y throw

```
val n = try { texto.toInt() }
        catch (e: Exception) { 0 }

fun retirar(cantidad: Double) {
    if (cantidad <= 0) {
        throw IllegalArgumentException("Inválida")
    }
}
```

✗ Mal — ConcurrentModificationException

```
for (p in personas) {
    if (p.edad < 18) personas.remove(p)
}
```

✓ Bien

```
personas.removeIf { it.edad < 18 }
```

→ No modifies una lista mientras la recorres con `for`: usa `removeIf`.



TRUCO DE EXAMEN

En un ejercicio de corregir código, resuelve primero los errores que impiden **compilar**. Después busca los de **ejecución** y, por último, los **lógicos**.

19 Android / Jetpack Compose

cómo leerlo y modificarlo en un examen

PASO 1

Dato

Identifica el estado: qué variable cambia.

PASO 2

Función

Crea o modifica la lógica que lo cambia.

PASO 3

Evento

Conéctala: `onClick`, `onValueChange` ...

PASO 4

Interfaz

Muestra el resultado leyendo el estado.

@Composable y Text

```
@Composable
fun Saludo(nombre: String) {
    Text(text = "Hola $nombre")
}
```

→ `@Composable` indica que la función construye interfaz. `Text` muestra texto.

Button

```
Button(onClick = {
    println("Pulsado")
}) {
    Text("Aceptar")
}
```

→ `onClick` contiene el código que se ejecuta al pulsar.

Estado: remember + mutableStateOf

```
var contador by remember { mutableStateOf(0) }

Button(onClick = { contador++ }) {
    Text("Contador: $contador")
}
```

→ Al cambiar un estado, Compose redibuja lo que lo usa (recomposición). `remember` lo conserva entre redibujados.

TextField

```
var texto by remember { mutableStateOf("") }

TextField(
    value = texto,
    onValueChange = { texto = it },
    label = { Text("Nombre") }
)

val edad = texto.toIntOrNull() ?: 0 // número
```

◀ LazyColumn — mostrar una lista

```
LazyColumn {
    items(personas) { persona ->
        Text(persona.nombre)
    }
}
```

→ `items` recorre la colección y pasa cada elemento al bloque.

◀ Botón que llama a una función

```
Button(onClick = {
    eliminarPersona("Ana")
}) {
    Text("Eliminar")
}
```

→ Una cosa es la **lógica** (la función) y otra la **interfaz** (el botón que la llama).

◀ Column y Row

```
Column(modifier = Modifier.padding(16.dp)) {
    Text("Arriba")
    Row {
        Text("Izquierda")
        Text("Derecha")
    }
}
```

→ `Column` apila en vertical; `Row`, en horizontal.

◀ Lista como estado

```
val personas = remember {
    mutableStateListOf<Persona>()
}
personas.add(Persona("Ana", 20)) // UI actualizada
// para cambiar un objeto, reemplázalo:
personas[0] = personas[0].copy(edad = 21)
```

→ Con un `mutableListOf` normal, Compose **no se entera** de los cambios; con `mutableStateListOf`, sí.

◀ Pantalla completa: añadir y listar

PLANTILLA

```
@Composable
fun PantallaPersonas() {
    val personas = remember { mutableStateListOf<Persona>() }
    var nombre by remember { mutableStateOf("") }

    Column(modifier = Modifier.padding(16.dp)) {
        TextField(value = nombre, onValueChange = { nombre = it }, label = { Text("Nombre") })
        Button(onClick = {
            if (nombre.isNotBlank()) {
                personas.add(Persona(nombre, 0))
                nombre = "" // limpia el campo
            }
        }) { Text("Añadir") }
        LazyColumn {
            items(personas) { persona -> Text("${persona.nombre} - ${persona.edad}") }
        }
    }
}
```



ERROR TÍPICO

«Property delegate must have a 'getValue(...)' method» al usar `by remember` → faltan los imports `androidx.compose.runtime.getValue` y `setValue`. Para `items(lista)` hace falta `androidx.compose.foundation.lazy.items`.



TRUCO DE EXAMEN

Si el proyecto ya declara la lista o el estado de otra forma (por ejemplo, en un ViewModel), sigue el patrón que haya planteado el profesor en lugar de cambiarlo.

Modo examen

Plantillas completas para copiar y adaptar, una tabla para decidir qué usar en cada caso, los métodos paso a paso para corregir y para añadir funcionalidades, y ejercicios resueltos.

plantilla CRUD

¿qué uso?

corregir en 8 pasos

añadir en 5 pasos

errores

ejercicios

20 Plantilla completa: lista de objetos

añadir · eliminar · buscar · modificar · listar

Datos, añadir, eliminar y buscar

PLANTILLA

```
data class Persona(
    val nombre: String,
    var edad: Int
)

val personas = mutableListOf<Persona>()

fun agregarPersona(nombre: String, edad: Int) {
    personas.add(Persona(nombre, edad))
}

fun eliminarPersona(nombre: String) {
    personas.removeIf { it.nombre == nombre }
}

fun buscarPersona(nombre: String): Persona? {
    return personas.find { it.nombre == nombre }
}

fun existePersona(nombre: String): Boolean {
    return personas.any { it.nombre == nombre }
}
```

Modificar, consultar y mostrar

PLANTILLA

```
fun modificarEdad(
    nombre: String,
    edad: Int
): Boolean {
    val p = buscarPersona(nombre) ?: return false
    p.edad = edad
    return true
}

fun contarMayores(): Int =
    personas.count { it.edad >= 18 }

fun obtenerMayores(): List<Persona> =
    personas.filter { it.edad >= 18 }

fun obtenerNombres(): List<String> =
    personas.map { it.nombre }

fun mostrarPersonas() {
    if (personas.isEmpty()) {
        println("No hay personas")
        return
    }
    for (p in personas) {
        println("${p.nombre} - ${p.edad}")
    }
}
```

Cómo adaptarla a tu ejercicio

EJERCICIO	CLASE	PROPIEDADES	CONDICIÓN TÍPICA
Tienda	Producto	nombre: String, precio: Double, stock: Int	it.precio > 2.0 · it.stock == 0
Clase	Alumno	nombre: String, nota: Double	it.nota >= 5
Biblioteca	Libro	titulo: String, autor: String, prestado: Boolean	!it.prestado
Cine	Pelicula	titulo: String, anio: Int, duracion: Int	it.anio > 2000

**ANTES DE COPIAR, 4 COMPROBACIONES**

① Nombre de la lista · ② tipo de objeto · ③ nombres de las propiedades · ④ tipo que devuelve la función. La estructura te sirve; los nombres deben coincidir con **tu** programa.

21 ¿Qué me piden? → Qué uso

tabla de decisión rápida

SI TE PIDEN...	USA	EJEMPLO
Añadir un elemento	<code>add</code>	<code>lista.add(Persona("Ana", 20))</code>
Eliminar por posición	<code>removeAt</code>	<code>lista.removeAt(2)</code>
Eliminar por valor / condición	<code>remove</code> · <code>removeIf</code>	<code>lista.removeIf { it.nombre == n }</code>
Buscar un objeto	<code>find</code>	<code>lista.find { it.nombre == n }</code>
Saber en qué posición está	<code>indexOfFirst</code>	<code>lista.indexOfFirst { it.nombre == n }</code>
Saber si existe	<code>any</code> · <code>in</code>	<code>lista.any { it.edad >= 18 }</code>
Comprobar que todos cumplen	<code>all</code>	<code>lista.all { it.nota >= 5 }</code>
Quedarte con varios	<code>filter</code>	<code>lista.filter { it.precio > 2.0 }</code>
Sacar una propiedad de todos	<code>map</code>	<code>lista.map { it.nombre }</code>
Contar	<code>count</code> · <code>contador</code>	<code>lista.count { it.edad >= 18 }</code>
Sumar / hacer la media	<code>sumOf</code> · <code>average</code>	<code>lista.sumOf { it.precio }</code>
El mayor / el menor	<code>maxByOrNull</code> · <code>minBy...</code>	<code>lista.maxByOrNull { it.nota }</code>
Ordenar	<code>sortedBy</code> · <code>...Descending</code>	<code>lista.sortedBy { it.nombre }</code>
Recorrer	<code>for (x in lista)</code>	<code>for (p in personas) println(p.nombre)</code>
Recorrer necesitando la posición	<code>indices</code> · <code>withIndex</code>	<code>for (i in lista.indices)</code>
Trabajar con una matriz	<code>dos for</code>	<code>matriz[fila][columna]</code>
Un menú de opciones	<code>do-while</code> + <code>when</code>	<code>when (opcion) { 1 -> ... }</code>
Un valor que puede faltar	<code>?.</code> · <code>?:</code>	<code>persona?.edad ?: 0</code>

22 Método para CORREGIR código

sigue este orden siempre

1 Tipos

¿Cada variable recibe un valor compatible con su tipo?

2 val / var

¿Se intenta cambiar una variable declarada con `val`?

3 Colecciones

Si hay `add` / `remove`, ¿es una `MutableList`?

4 Índices

Revisa todos los `[i]`. Busca especialmente `0..size`.

5 Null

Busca `String?`, `find`, objetos opcionales y accesos a propiedades.

6 Funciones

Parámetros, tipos de retorno, `return` ... ¿y se llaman?

7 Lógica

¿Las condiciones hacen realmente lo que pide el enunciado?

8 Android

Si es Compose: estados, eventos y listas.

✗ Código con errores

```
val nombres = listOf("Ana", "Luis")
nombres.add("Marta")           // ✗ 1
for (i in 0..nombres.size) {   // ✗ 2
    println(nombres[i])
}
```

Problema 1: `nombres` es `List` pero se usa `add` → debe ser `MutableList`.

Problema 2: `0..size` incluye `size` → usa `indices` o `until`.

✓ Corregido

```
val nombres = mutableListOf("Ana", "Luis")
nombres.add("Marta")
for (i in nombres.indices) {
    println(nombres[i])
}
```

→ Evita ambos errores y, además, `indices` hace que el recorrido siga siendo seguro si cambia el tamaño de la lista.

23 Método para AÑADIR una funcionalidad

no escribas código al azar

PASO 1

¿Qué dato necesito?

Para eliminar una persona necesito su nombre.



PASO 2

¿Dónde están los datos?

Normalmente en una `MutableList`.



PASO 3

¿Qué función lo hace?

Crea una función clara.



PASO 4

¿Quién la llama?

`main`, el menú o un `Button`.



PASO 5

¿Cómo se ve?

La salida o la UI muestra el estado nuevo.

◀ Ejemplo: añadir eliminación por nombre

```
data class Persona(  
    val nombre: String,  
    val edad: Int  
)  
  
val personas = mutableListOf(  
    Persona("Ana", 20),  
    Persona("Luis", 17)  
)  
  
fun eliminarPersona(nombre: String) {  
    personas.removeIf {  
        it.nombre == nombre  
    }  
}  
  
eliminarPersona("Ana")
```

→ La función recibe el nombre, busca las personas cuyo nombre coincide y las elimina. La funcionalidad queda separada del resto.

◀ Si fuera un botón (Compose)

```
Button(onClick = {  
    eliminarPersona("Ana")  
}) {  
    Text("Eliminar")  
}
```

→ El evento `onClick` llama a la función. Lógica e interfaz, separadas.

◀ Si fuera un menú (consola)

```
when (opcion) {  
    1 -> agregar()  
    2 -> mostrar()  
    3 -> { // ← nueva opción  
        print("Nombre a eliminar: ")  
        eliminarPersona(readln())  
    }  
    0 -> println("Adiós")  
}
```



ERROR TÍPICO

Escribir la función nueva... y no llamarla nunca. Comprueba que tu funcionalidad se usa desde algún sitio: `main`, el menú o un evento de la interfaz.

24 Errores que debes reconocer de memoria

mensaje → significado → solución

MENSAJE	QUÉ SIGNIFICA	CÓMO SE ARREGLA
Unresolved reference: x	Kotlin no encuentra ese nombre (variable, función o propiedad).	Revisa cómo está escrito, mayúsculas e <code>import</code> .
Unresolved reference: add	Usas <code>add</code> sobre una <code>List</code> (solo lectura).	Declárala con <code>mutableListOf</code> .
Type mismatch: inferred type is String but Int was expected	El tipo que tienes no coincide con el esperado.	Convierte: <code>toInt()</code> , <code>toString()</code> ...
Val cannot be reassigned	Intentas cambiar una variable <code>val</code> .	Cámbiala a <code>var</code> .
Variable 'x' must be initialized	Usas una variable antes de darle valor.	Dale un valor inicial.
Only safe (?.) or non-null asserted (!!.) calls are allowed...	Usas un tipo nullable como si no lo fuera.	<code>?.</code> , <code>?:</code> o <code>if (x != null)</code> .
Smart cast to 'X' is impossible...	Es una propiedad <code>var</code> que podría cambiar entre la comprobación y el uso.	Cópiala a un <code>val</code> local o usa <code>?.let</code> .
A 'return' expression required in a function with a block body	La función declara un tipo de retorno pero algún camino no devuelve nada.	Añade <code>return</code> en todos los caminos.
No value passed for parameter 'x'	Llamas a una función o constructor con argumentos de menos o mal ordenados.	Revisa número, orden y tipo de los parámetros.
Expecting ')' · Expecting '}'	Problema de sintaxis.	Cuenta paréntesis, llaves y comas.
IndexOutOfBoundsException	Ejecución: accedes a una posición que no existe.	<code>indices</code> o <code>until</code> .
NullPointerException	Ejecución: usas un <code>null</code> como si fuera un objeto (casi siempre por <code>!!</code>).	<code>?.</code> / <code>?:</code> .

MENSAJE	QUÉ SIGNIFICA	CÓMO SE ARREGLA
<code>NumberFormatException</code>	Ejecución: <code>toInt()</code> sobre un texto que no es un número.	<code>toIntOrNull()</code> o <code>try/catch</code> .
<code>ConcurrentModificationException</code>	Ejecución: modificas la lista mientras la recorres con <code>for</code> .	<code>removeIf { ... }</code> .
Property delegate must have a 'getValue(...)' method	Compose: faltan imports para <code>by remember</code> .	Importa <code>getValue</code> y <code>setValue</code> .

25 Ejercicios tipo examen

intenta resolverlos antes de mirar

EJERCICIO A Corrige Encuentra los dos errores.

Enunciado

```
data class Producto(
    val nombre: String,
    val precio: Double
)
val productos = listOf(
    Producto("Pan", 1.20),
    Producto("Queso", 4.50)
)
productos.add(Producto("Leche", 1.50))
for (i in 0..productos.size) {
    println(productos[i].nombre)
}
```

Solución explicada

```
val productos = mutableListOf(      // ✓ 1
    Producto("Pan", 1.20),
    Producto("Queso", 4.50)
)
productos.add(Producto("Leche", 1.50))
for (i in productos.indices) {      // ✓ 2
    println(productos[i].nombre)
}
```

- 1: `listOf` → `mutableListOf` porque necesitamos `add`.
- 2: `indices` para no salirnos de la lista.

EJERCICIO B Añade funcionalidad Una función que devuelva los productos de más de 2 €.

Solución con `filter`

```
fun productosCaros(): List<Producto> {
    return productos.filter {
        it.precio > 2.0
    }
}
```

- `filter` encaja porque queremos **seleccionar** algunos y mantenerlos en una lista.

Solución con `for`

```
fun productosCaros(): List<Producto> {
    val caros = mutableListOf<Producto>()
    for (p in productos) {
        if (p.precio > 2.0) caros.add(p)
    }
    return caros
}
```

EJERCICIO C Corrige La media sale mal y a veces el programa falla. Hay tres problemas.

Enunciado

```
fun media(notas: List<Int>): Int {
    var suma = 0
    for (i in 0..notas.size) {
        suma += notas[i]
    }
    return suma / notas.size
}
```

Solución explicada

```
fun media(notas: List<Int>): Double {      // ✓ 2
    if (notas.isEmpty()) return 0.0        // ✓ 3
    var suma = 0
    for (nota in notas) {                  // ✓ 1
        suma += nota
    }
    return suma.toDouble() / notas.size    // ✓ 2
}
```

- 1: `0..size` se sale de la lista. 2: división entera: hay que devolver `Double`. 3: con la lista vacía se dividiría entre 0.

EJERCICIO D Matrices Devuelve una lista con la suma de cada fila.

◀ Solución

```
fun sumaPorFila(m: Array<IntArray>): List<Int> {  
    val sumas = mutableListOf<Int>()  
    for (fila in m.indices) {  
        var suma = 0  
        for (col in m[fila].indices) {  
            suma += m[fila][col]  
        }  
        sumas.add(suma)  
    }  
    return sumas  
}  
// atajo: m.map { it.sum() }
```

EJERCICIO E Null ¿Por qué no compila?

◀ Enunciado

```
val alumno = alumnos.find { it.nombre == "Pepe" }  
println(alumno.nota)    // X no compila
```

◀ Solución

```
println(alumno?.nota ?: "No existe")
```

→ `find` devuelve `Alumno?` : puede ser `null` y hay que tratarlo.

Repaso final

Todo Kotlin del examen en dos páginas, la checklist de los dos minutos antes de entregar y el orden recomendado para estudiar.

chuleta

checklist

plan de estudio

26 Chuleta ultrarrápida

todo en un vistazo

01 VARIABLES

```
val x = 5           // fija
var y = 5           // cambia
val n: Int = 1
val d: Double = 2.5
val s: String = "Hola"
val c: Char = 'A'
val b: Boolean = true
```

05 BUCLES

```
for (i in 1..5)           // 1..5
for (i in 0 until 5)      // 0..4
for (i in 0..<5)          // 0..4
for (i in 5 downTo 1)     // 5..1
for (i in 0..10 step 2)   // 0,2,...
for (x in lista) { }
for (i in lista.indices) { }
for ((i, x) in l.withIndex())
while (cond) { }
do { } while (cond)
repeat(3) { }
break · continue
```

09 LISTAS

```
val l = listOf("A", "B") // fija
val m = mutableListOf<String>()
m.add("C")               m.add(0, "Z")
m.remove("A")             // por valor
m.removeAt(0)             // por posición
m[0] = "X"               m.size
m.indices                m.isEmpty()
"X" in m                 m.indexOf("X")
m.addAll(l)              m.clear()
m.sort()                 // la misma
m.sorted()               // una nueva
```

12 NULL

```
var s: String? = null
s?.length               // null si s es null
s ?: "defecto"          // Elvis
s?.length ?: 0
s!!.length              // X NPE si null
if (s != null) s.length
s?.let { println(it) }
```

02 OPERADORES

```
10 / 3                 // 3 (entera)
10 / 3.0               // 3.333
10 % 3                 // 1 (resto)
== != > < >= <=
&& || !
x += 1; x++
"25".toInt()
"3.5".toDouble()
"abc".toIntOrNull()    // null
5.toString()
"%2f".format(x)
```

06 FUNCIONES

```
fun sumar(a: Int, b: Int): Int {
    return a + b
}
fun doble(n: Int) = n * 2
fun hola(n: String = "tú") { }
fun buscar(): Persona? { }
```

07 STRINGS

```
s.length               s[0]
s.uppercase()          s.lowercase()
s.trim()               s.reversed()
s.contains("a")
s.replace("a", "b")
s.split(" ")
s.substring(0, 3)
s.indexOf("a")          // -1 si no
s.startsWith("H")
s.isBlank()            s.isEmpty()
s.equals(t, ignoreCase = true)
"${p.nombre} tiene $edad"
```

11 MATRICES

```
val m = Array(3) { IntArray(3) }
m[f][c] = 5
m.size                 // filas
m[0].size              // columnas
for (f in m.indices) {
    for (c in m[f].indices) {
        println(m[f][c])
    }
}
for (i in m.indices) d += m[i][i]
```

03 CONSOLA

```
println("Hola $nombre")
print("sin salto")
val t = readln()
val n = readln().toInt()
val m = readln().toIntOrNull()
```

04 CONDICIONES

```
if (x > 0) { } else { }
val m = if (a > b) a else b
when (op) {
    1 -> agregar()
    2, 3 -> mostrar()
    in 4..6 -> otro()
    else -> println("No")
}
val r = when {
    nota >= 5 -> "Aprobado"
    else -> "Suspenso"
}
```

08 ARRAYS

```
val a = arrayOf(1, 2, 3)
val b = IntArray(5)    // ceros
a[0] = 9
a.size                a.indices
a.sum()               a.average()
a.maxOrNull()
a.contentToString()   // ver
```

10 COLECCIONES

```
l.find { cond }         // T?
l.indexOfFirst { cond } // -1
l.filter { cond }       // List
l.map { it, prop }      // List
l.any { cond }          // Bool
l.all { cond }          // Bool
l.none { cond }         // Bool
l.count { cond }        // Int
l.sumOf { it, precio }
l.maxByOrNull { it, x } // T?
l.sortedBy { it, x }
l.sortedByDescending { it, x }
l.forEach { println(it) }
m.removeIf { cond }
l.joinToString(", ")
```

13 CLASES

```
data class P(  
    val nombre: String,  
    var edad: Int  
)  
val p = P("Ana", 20)  
p.edad = 21 // var  
val q = p.copy(edad = 30)  
open class A { open fun f() {} }  
class B : A() {  
    override fun f() {}  
}
```

14 MAPS Y SETS

```
val mp = mutableMapOf("a" to 1)  
mp["b"] = 2 // poner  
mp["a"] // Int?  
mp.getOrElse("z", 0)  
mp.remove("a")  
"a" in mp // ¿clave?  
for ((k, v) in mp) { }  
val st = mutableSetOf(1, 2)  
1.distinct() // sin repes
```

16 EXCEPCIONES

```
try {  
    val n = t.toInt()  
} catch (e: Exception) {  
    println("No es número")  
}  
throw IllegalArgumentException("...")
```

15 COMPOSE

```
var x by remember {  
    mutableStateOf(0)  
}  
val lista = remember {  
    mutableStateListOf<P>()  
}  
Button(onClick = { x++ }) {  
    Text("Sumar")  
}  
TextField(  
    value = t,  
    onChange = { t = it }  
)  
LazyColumn {  
    items(lista) { Text(it.nombre) }  
}  
Column { } Row { }
```

27 Checklist final y plan de estudio

Los 2 minutos antes de entregar

- | | |
|---|--|
| ☐ ¿El código compila? Revisa primero sintaxis y tipos. | ☐ ¿He confundido <code>=</code> con <code>==</code> ? |
| ☐ ¿He usado <code>val</code> donde necesito cambiar el valor? | ☐ ¿La lista necesita ser <code>MutableList</code> ? |
| ☐ ¿He revisado todos los índices? ¿Algún <code>0..lista.size</code> ? | ☐ ¿ <code>remove</code> (valor) o <code>removeAt</code> (posición)? |
| ☐ ¿Hay alguna división entera donde quería decimales? | ☐ ¿Puede alguna variable ser <code>null</code> ? ¿Y lo que devuelve <code>find</code> ? |
| ☐ ¿Las funciones reciben y devuelven lo que toca, con su <code>return</code> ? | ☐ ¿Uso <code>"\${obj.prop}"</code> con llaves dentro de los textos? |
| ☐ ¿Mi funcionalidad nueva se llama desde algún sitio? | ☐ Si es Compose, ¿el dato que cambia es un estado? |
| ☐ ¿Puedo explicar oralmente cada cambio que he hecho? | ☐ ¿He probado el caso «lista vacía» y el caso «no encontrado»? |

Orden recomendado de estudio

1 Variables y tipos temas 01-02	2 if / when tema 04	3 Bucles tema 05	4 Arrays tema 08	5 Listas tema 09	6 Matrices tema 13
7 Funciones tema 06	8 Null safety tema 15	9 Clases y data class temas 16-17	10 filter / map / find temas 10-12	11 Android / Compose tema 19	12 Simulacros temas 22-25

Tu objetivo no es memorizar 500 líneas de Kotlin. Es reconocer patrones.

Si dominas List / MutableList, arrays y matrices, bucles, funciones, null safety, clases y condicionales, ya tienes una base muy sólida para este examen. ¡Mucha suerte!